

UNIT10: ADVANCED SPRITE

【学習要項】

□Completion

【演習手順】

1. テクスチャのロードをモジュール化する(texture.h, texture.cpp)

※sprite, sprite_batch クラスのコンストラクタで、テクスチャロードのコードをこの関数を使ったものに書き換える

```
1: #include <WICTextureLoader.h>
2: using namespace DirectX;
3:
4: #include <wrl.h>
5: using namespace Microsoft::WRL;
6:
7: #include <string>
8: #include <map>
9: using namespace std;
10:
11: static map<wstring, ComPtr<ID3D11ShaderResourceView>> resources;
12: HRESULT load_texture_from_file(ID3D11Device* device, const wchar_t* filename,
13:     ID3D11ShaderResourceView** shader_resource_view, D3D11_TEXTURE2D_DESC* texture2d_desc)
14: {
15:     HRESULT hr{ S_OK };
16:     ComPtr<ID3D11Resource> resource;
17:
18:     auto it = resources.find(filename);
19:     if (it != resources.end())
20:     {
21:         *shader_resource_view = it->second.Get();
22:         (*shader_resource_view)->AddRef();
23:         (*shader_resource_view)->GetResource(resource.GetAddressOf());
24:     }
25:     else
26:     {
27:         hr = CreateWICTextureFromFile(device, filename, resource.GetAddressOf(), shader_resource_view);
28:         _ASSERT_EXPR(SUCCEEDED(hr), hr_trace(hr));
29:         resources.insert(make_pair(filename, *shader_resource_view));
30:     }
31:
32:     ComPtr<ID3D11Texture2D> texture2d;
33:     hr = resource.Get()->QueryInterface<ID3D11Texture2D>(texture2d.GetAddressOf());
34:     _ASSERT_EXPR(SUCCEEDED(hr), hr_trace(hr));
35:     texture2d->GetDesc(texture2d_desc);
36:
37:     return hr;
38: }
39: void release_all_textures()
40: {
41:     resources.clear();
42: }
```

2. シェーダーファイルのロードをモジュール化する(shader.h, shader.cpp)

※sprite, sprite_batch クラスのコンストラクタで、シェーダーファイルロードのコードをこの関数を使ったものに書き換える

※必要に応じてヘッダファイルをインクルードする

```
1: HRESULT create_vs_from_cso(ID3D11Device* device, const char* cso_name, ID3D11VertexShader** vertex_shader,
2:     ID3D11InputLayout** input_layout, D3D11_INPUT_ELEMENT_DESC* input_element_desc, UINT num_elements)
3: {
4:     FILE* fp{ nullptr };
5:     fopen_s(&fp, cso_name, "rb");
6:     _ASSERT_EXPR_A(fp, "CSO File not found");
7:
8:     fseek(fp, 0, SEEK_END);
9:     long cso_sz{ ftell(fp) };
10:    fseek(fp, 0, SEEK_SET);
11:
12:    unique_ptr<unsigned char[]> cso_data{ make_unique<unsigned char[]>(cso_sz) };
```

```
13: fread(cso_data.get(), cso_sz, 1, fp);
14: fclose(fp);
15:
16: HRESULT hr{ S_OK };
17: hr = device->CreateVertexShader(cso_data.get(), cso_sz, nullptr, vertex_shader);
18: _ASSERT_EXPR(SUCCEEDED(hr), hr_trace(hr));
19:
20: if (input_layout)
21: {
22:     hr = device->CreateInputLayout(input_element_desc, num_elements,
23:         cso_data.get(), cso_sz, input_layout);
24:     _ASSERT_EXPR(SUCCEEDED(hr), hr_trace(hr));
25: }
26:
27: return hr;
28: }
29:
30: HRESULT create_ps_from_cso(ID3D11Device* device, const char* cso_name, ID3D11PixelShader** pixel_shader)
31: {
32:     FILE* fp{ nullptr };
33:     fopen_s(&fp, cso_name, "rb");
34:     _ASSERT_EXPR_A(fp, "CSO File not found");
35:
36:     fseek(fp, 0, SEEK_END);
37:     long cso_sz{ ftell(fp) };
38:     fseek(fp, 0, SEEK_SET);
39:
40:     unique_ptr<unsigned char[]> cso_data{ make_unique<unsigned char[]>(cso_sz) };
41:     fread(cso_data.get(), cso_sz, 1, fp);
42:     fclose(fp);
43:
44:     HRESULT hr{ S_OK };
45:     hr = device->CreatePixelShader(cso_data.get(), cso_sz, nullptr, pixel_shader);
46:     _ASSERT_EXPR(SUCCEEDED(hr), hr_trace(hr));
47:
48:     return hr;
49: }
```

3. COM オブジェクトを ComPtr スマートポインターテンプレートを使った変数宣言に変更する

①必要なヘッダーファイル

```
#include <wrl.h>
```

②変数宣言(例)

```
ID3D11Device* device;    ->    Microsoft::WRL::ComPtr<ID3D11Device> device;
```

③関数の引数に渡す場合(例)

```
device        ->    device.Get()
&device       ->    device.GetAddressOf()
```

④メンバ関数の呼び出しは同じ

⑤Release メンバ関数の呼び出しは不要

4. sprite, sprite_batch オブジェクト等をスマートポインターを使った変数宣言に変更する

5. sprite, sprite_batch クラスの render メンバ関数をオーバーロードする

※画面上の描画位置とサイズの指定のみでテクスチャ全体を描画する

```
void render(ID3D11DeviceContext* immediate_context, float dx, float dy, float dw, float dh);
```

6. sprite_batch オブジェクト描画時、別途ロードしたピクセルシェーダーに差替える仕組みを考える

UNIT10: ADVANCED SPRITE

7. sprite クラスにフォント画像ファイルを使用し任意の文字列を画面に出力する機能(textout)を追加する

※フォント画像ファイルはアスキーコード順に16 x 16の文字が配置された画像ファイル

※フォント画像ファイルは fonts フォルダのものを使用する

```
1: void sprite::textout(ID3D11DeviceContext* immediate_context, std::string s,
2:   float x, float y, float w, float h, float r, float g, float b, float a)
3: {
4:   float sw = static_cast<float>(texture2d_desc.Width / 16);
5:   float sh = static_cast<float>(texture2d_desc.Height / 16);
6:   float carriage = 0;
7:   for (const char c : s)
8:   {
9:     render(immediate_context, x + carriage, y, w, h, r, g, b, a, 0,
10:      sw * (c & 0x0F), sh * (c >> 4), sw, sh);
11:     carriage += w;
12:   }
13: }
```

8. Release モードでビルド・実行できることを確認する

9. 2D ゲームを開発することでプログラムの品質・性能向上を目指す

【評価項目】

- ☐ スプライトクラスの改良
- ☐ 2D ゲームの作成